



North South University

Course name : CSE 225
Section : 12
Semester : Summer 24
HW : 06
Date : 30-09-2024

Submitted To :

Dr. Md. Ahsan Habib (AHbb)

Submitted by :

NAME : Mohammad Tushar Abdullah

ID : 2323766042

Ans. to the question. no-01

a)

The maximum number of nodes upto a height h is given by the sum of a geometric series: $1 + 2 + 4 + \dots + 2^h = 2^{h+1} - 1$

Complete binary tree with height h has at most $2^{h+1} - 1$ nodes

$$n \leq 2^{h+1} - 1$$

$$n = 2^{h+1} - 1$$

$$\Rightarrow 2^{h+1} = n + 1$$

$$\Rightarrow \log_2(2^{h+1}) = \log_2(n+1)$$

$$\Rightarrow h+1 = \log_2(n+1)$$

$$\Rightarrow h = \log_2(n+1) - 1$$

$$\therefore O(\log n)$$

Therefore the complete binary tree with n nodes has height at most $O(\log n)$

⑥

Given that,

A complete binary tree has 127 nodes.

$$n = 127$$

maximum possible height

$$n = 2^{h+1} - 1.$$

$$\Rightarrow 127 = 2^{h+1} - 1.$$

$$\Rightarrow 128 = 2^{h+1}$$

$$\therefore h = 6.$$

$\therefore$ Maximum possible height is 6.

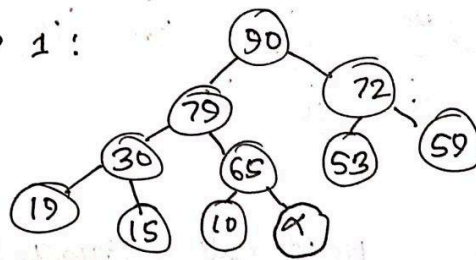
Ans. to the question no-02

9)

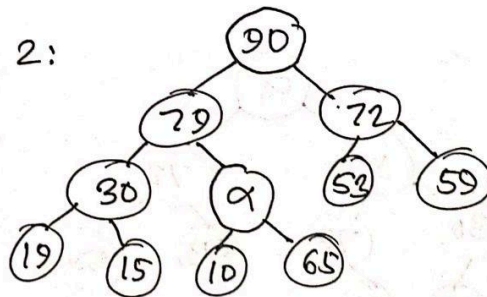
We need to remove the node with value 23 from the heap.

- ① change the priority of the element 23 to α
- ② Then let it sift up.
- ③ Then extract maximum.

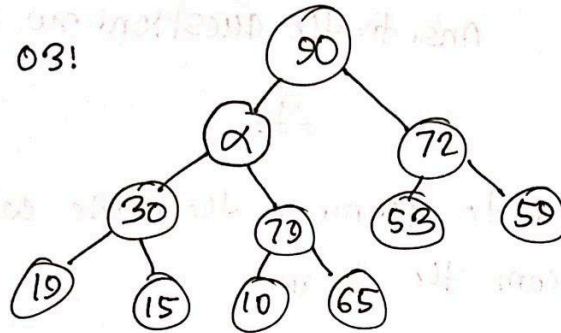
Step 1:



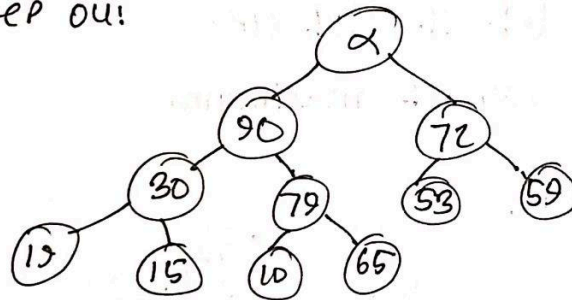
Step 2:



step 03:

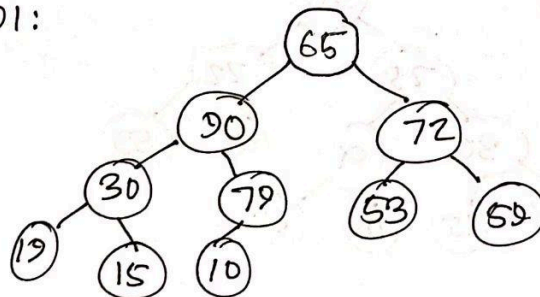


step 04:



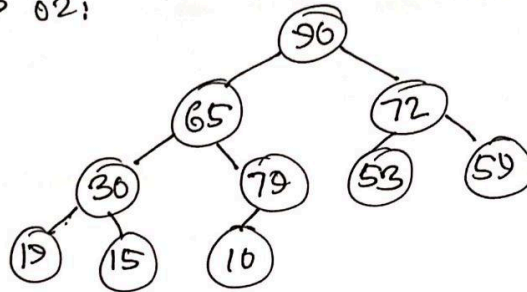
NOW can extract Max

step 01:

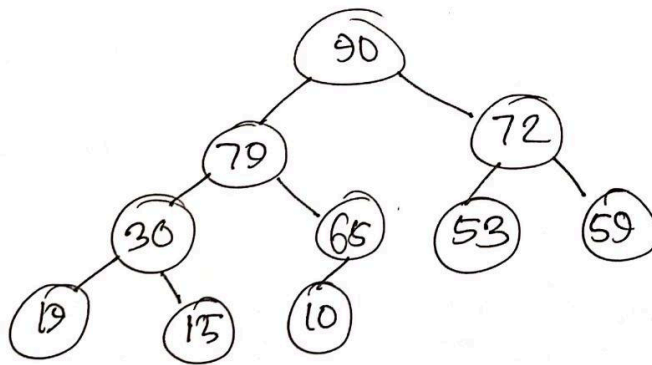


replace α with 65 leaf node 65

step 02:



step 03:



This is our required binary heap.

(b)

A complete binary tree has a height of $O(\log n)$.

When we have to remove a node from the heap and restore the heap property, we need to make the node to infinity. Then we have to sift down until it goes to the root node. After that we have to extract max. Then we remove the root node and replace it with the last leaf node of the heap. This takes $O(1)$ time complexity because accessing the last element and the root node takes constant time. Then we sift down until it makes the property of a heap. As we have to sift down, the operation may take at most equal to the height of the tree. As the height of the tree is $O(\log n)$, sift down operations have time complexity $O(\log n)$.

$$\begin{aligned} & O(1) + O(\log n) \\ = & O(\log n) \end{aligned}$$

Therefore, the overall time complexity of removing a node and restoring the heap property is $O(\log n)$.